

Mini-SOC maison : tests de détection mappés sur MITRE ATT&CK

Émulation de comportements d'attaquant avec Atomic Red Team sur un lab isolé, et vérification de ce qu'un SIEM Wazuh voit réellement, sur Windows et sur Linux.

Wazuh 4.14

Sysmon

Atomic Red Team

MITRE ATT&CK

Proxmox

Auteur	collapsedbanana
Date des tests	3 octobre 2026
Version du document	1.0
Périmètre	Lab personnel isolé (réseau 10.10.10.0/24) : 1 SIEM, 1 poste Windows 11, 1 serveur Ubuntu 24.04
Type d'exercice	Purple team : on joue l'attaquant et le défenseur pour mesurer la détection

Diffusion : publique (portfolio GitHub). Toutes les machines testées m'appartiennent et sont isolées du réseau domestique. Aucun identifiant réel n'apparaît dans ce document.

Sommaire

1 Résumé

Ce qu'il faut retenir en une page

2 Contexte et notions de base

SOC, SIEM, agent, Sysmon, MITRE ATT&CK, Atomic Red Team, règles et niveaux

3 Architecture du lab

Les machines, le réseau, le chemin des logs

4 Méthodologie

Le cycle de test, la grille de verdict, les règles du jeu

5 Résultats détaillés, test par test

5 techniques Windows, 2 tests Linux, 1 faux positif

6 Leçons et recommandations

7 Limites et prochaines étapes

A Annexe A : glossaire

B Annexe B : la règle 100100 expliquée ligne par ligne

C Annexe C : réutiliser ce rapport comme modèle

COMMENT LIRE CE RAPPORT

Un lecteur pressé (un recruteur, un manager) lit la **section 1** : elle tient en une page et donne le résultat. Un lecteur technique va directement à la **section 5**. Si un mot vous bloque, il est défini dans la **section 2** ou dans le **glossaire** (annexe A).

Les encadrés ont toujours le même sens : **vert** pour ce qu'il faut retenir, **orange** pour un piège ou une limite, **bleu** pour une explication ou un choix.

1 Résumé

La question posée : si un attaquant se comporte sur mes machines comme le font les vrais attaquants, est-ce que mon SIEM le voit, et est-ce qu'il le signale avec le bon niveau d'urgence ?

Ce qui a été fait : sur un lab isolé, j'ai rejoué 7 comportements d'attaquant connus, chacun référencé dans la matrice MITRE ATT&CK. Pour chacun, j'ai regardé ce que le SIEM Wazuh remontait, puis j'ai corrigé ce qui manquait et rejoué le test pour le prouver.

7 comportements testés (5 Windows, 2 Linux)	5 tactiques MITRE couvertes	7/7 visibles dans le SIEM après corrections	2 trous de détection trouvés et corrigés
--	--	--	---

#	Technique MITRE	Tactique	Avant correction	Après
1	T1136.001 Création d'un compte local	Persistence	Déecté (niv. 12)	Inchangé
2	T1053.005 Tâche planifiée	Persistence, Execution	Mal qualifié (niv. 3)	Règle 100100 (niv. 10)
3	T1547.001 Clé de registre Run	Persistence, Privilege Esc.	Déecté (niv. 6)	Inchangé
4	T1003.001 Mémoire de LSASS	Credential Access	Invisible au SIEM	Canal Defender collecté
5	T1070.001 Effacement des journaux	Defense Evasion	Déecté	Inchangé
L1	T1110 Force brute SSH	Credential Access	Déecté	Inchangé
L2	Création de compte vue par le FIM	Persistence	Déecté (niv. 7)	Inchangé

LES TROIS CHOSES À RETENIR

- **La détection native de Wazuh couvre déjà beaucoup** : 5 tests sur 7 étaient correctement détectés sans rien toucher.
- **Deux trous de nature différente** : une détection qui existait mais avec un niveau trop bas (corrigée par une règle personnalisée), et une source de logs qui n'était pas collectée du tout (corrigée par la configuration de l'agent).
- **Une recherche vide ne prouve rien** : plusieurs fois, l'événement existait mais la recherche était mal posée (casse, filtres restés actifs). C'est la leçon la plus utile du travail en SOC.

2 Contexte et notions de base

Cette section explique le vocabulaire nécessaire pour lire la suite. Si vous connaissez déjà ces notions, passez à la section 3.

2.1 Qu'est-ce qu'un SOC, et pourquoi en monter un chez soi ?

Un **SOC** (Security Operations Center) est l'équipe qui surveille en continu le système d'information d'une organisation pour repérer les attaques et y réagir. Son outil central est le **SIEM**. Monter un mini-SOC chez soi permet de faire ce que fait un analyste au quotidien : collecter des logs, lire des alertes, distinguer un vrai problème d'un faux positif, et améliorer les règles.

2.2 Les briques utilisées

Brique	Rôle, en une phrase	Analogie
SIEM (Wazuh)	Reçoit les logs de toutes les machines, les compare à des règles et lève des alertes. Il les stocke aussi hors de portée de l'attaquant.	La salle de vidéosurveillance d'un immeuble.
Agent	Petit programme installé sur chaque machine surveillée, qui lit les journaux locaux et les envoie au SIEM.	La caméra dans chaque couloir.
Sysmon	Outil gratuit de Microsoft qui ajoute à Windows des journaux très détaillés : chaque programme lancé, avec sa ligne de commande, son parent, ses empreintes.	Une caméra haute définition à la place d'une webcam.
MITRE ATT&CK	Catalogue public des comportements d'attaquants observés dans la réalité, rangés par <i>tactique</i> (le but) et <i>technique</i> (le moyen).	Le dictionnaire des modes opératoires des cambrioleurs.
Atomic Red Team	Bibliothèque de petits tests, chacun reproduisant une technique ATT&CK de façon contrôlée et réversible (option <code>-Cleanup</code>).	Un exercice d'alarme incendie : on simule pour vérifier que l'alarme sonne.

TECHNIQUE ATT&CK OU VULNÉRABILITÉ (CVE) : NE PAS CONFONDRE

Une **CVE** est une faille dans un logiciel précis (un bug qu'on corrige avec une mise à jour). Une **technique ATT&CK** est un comportement : créer un compte, planifier une tâche, effacer des journaux. Les tests de ce rapport n'exploitent aucune faille : ils utilisent des fonctions normales de Windows et de Linux, comme le ferait un attaquant déjà présent. C'est pour cela que la détection repose sur l'observation du comportement, pas sur un correctif.

2.3 Comment une alerte naît dans Wazuh

Le chemin est toujours le même, et le comprendre explique la plupart des surprises rencontrées pendant les tests :

1. **Un événement se produit** sur la machine (un programme est lancé, un fichier change).
2. **Il est journalisé** localement, si un journal le capte : Security, Sysmon, Defender, journald...
3. **L'agent l'envoie** au SIEM, à condition que ce journal figure dans sa configuration (`ossec.conf`).
4. **Le décodeur** découpe le texte en champs (utilisateur, commande, fichier...).

5. **Les règles** comparent ces champs à des conditions. Si une règle correspond, une **alerte** est créée avec un **niveau** de 0 à 15.
6. **Seules les alertes sont indexées** et visibles dans le dashboard.

CONSÉQUENCE PRATIQUE

Si une étape manque, rien n'apparaît, sans message d'erreur. Le test 4 (LSASS) est tombé dans le trou de l'étape 3 : l'événement existait sur le poste, mais l'agent ne lisait pas ce journal.

2.4 Le niveau d'une règle

Niveau	Sévérité affichée	Ce que ça veut dire pour l'analyste
0 à 6	Faible	Information, bruit de fond. On ne regarde pas une par une.
7 à 11	Moyenne	À examiner : changement notable (fichier système modifié, persistance probable).
12 à 14	Haute	À traiter vite : changement de privilèges, action de l'antivirus.
15	Critique	Urgence.

Choisir un niveau est un **compromis entre couverture et bruit** : une règle large déclenchée cent fois par jour doit rester basse, sinon les analystes finissent par ignorer toutes les alertes. Une règle précise, donc rare, peut être plus haute.

3 Architecture du lab

Tout tourne sur un seul PC sous l'hyperviseur **Proxmox**. Les machines testées sont sur un réseau virtuel isolé (`vmbr1`, 10.10.10.0/24), séparé du réseau de la maison : elles sortent sur Internet par NAT, mais rien du réseau domestique ne peut les joindre directement.

Hôte Proxmox (192.168.1.15) · réseau isolé vmbr1 10.10.10.0/24 · passerelle 10.10.10.1

wazuh · 10.10.10.10
SIEM tout-en-un : manager, indexeur, dashboard
Reçoit sur 1514/TCP (logs) et 1515/TCP (inscription)

win11 · 10.10.10.20
Windows 11 Entreprise (évaluation)
Sysmon + agent 001 + Atomic Red Team

ubuntu-cible · 10.10.10.30
Ubuntu Server 24.04
Agent 002 (journald, FIM)

Agents → manager : logs chiffrés sur 1514/TCP · Administrateur → lab : tunnel SSH par l'hôte Proxmox

Machine	Rôle	Ce qu'elle apporte au test
wazuh	SIEM	Collecte, règles, alertes, dashboard, tags MITRE et PCI DSS
win11	Poste de travail cible	Journal Security, Sysmon (config SwiftOnSecurity), Defender ; c'est là qu'Atomic Red Team joue l'attaquant
ubuntu-cible	Serveur Linux cible	Logs SSH via journald, contrôle d'intégrité des fichiers (FIM) sur <code>/etc</code>

3.1 Accès administrateur : le tunnel SSH

Comme le lab est isolé, on ne peut pas ouvrir le dashboard ou le Bureau à distance directement depuis le PC. On crée un **tunnel SSH** vers l'hôte Proxmox : un port local du PC (par exemple 13389) est relié, à travers SSH, au port 3389 de la VM Windows. Le client RDP se connecte alors à `localhost:13389`. Le nom de la machine Windows ne fonctionne pas, parce que le PC ne voit pas le réseau du lab : seul le tunnel y mène.

Mini-SOC maison : rapport de tests de détection

Page 5 / 26

4 Méthodologie

4.1 Le cycle appliqué à chaque test

Étape	Action	Question à laquelle elle répond
1	Choisir	Quelle technique ATT&CK, et pourquoi elle est réaliste pour cette machine ?
2	Émuler	Lancer le test Atomic correspondant, noter l'heure exacte.
3	Observer	Qu'est-ce qui a été journalisé sur le poste ? Qu'est-ce qui est arrivé dans le SIEM ?
4	Qualifier	L'alerte existe-t-elle, a-t-elle le bon niveau, le bon tag MITRE, assez de contexte pour un analyste ?
5	Corriger et rejouer	S'il y a un trou : règle ou collecte, puis on relance le même test pour prouver la correction.
6	Nettoyer	Remettre la machine dans son état initial (-Cleanup , suppression du compte de test).

4.2 La grille de verdict

Verdict	Signification
Déecté	Une alerte existe, avec un niveau et un contexte suffisants pour qu'un analyste réagisse.
Mal qualifié	Une alerte existe mais elle est noyée : niveau trop bas, titre générique, pas de lien avec la technique.
Non visible	Rien dans le SIEM, alors que l'action a bien eu lieu.
Corrigé	Le trou a été comblé et le test rejoué le prouve.

4.3 Règles du jeu

- Toutes les machines m'appartiennent et sont sur un réseau isolé ; aucun système tiers n'est touché.
- Les tests Atomic sont publics, documentés et réversibles ; chaque test est nettoyé après usage.
- L'antivirus Defender reste actif sur le poste, sauf une exclusion limitée au dossier d'Atomic Red Team pour pouvoir l'installer. Le test 4 montre d'ailleurs Defender en action.
- On mesure d'abord la détection **native**, sans rien modifier, avant toute correction : sinon on ne sait pas ce que la correction apporte.

5 Résultats détaillés, test par test

Chaque test suit la même structure : le **test** lancé, ce que les **logs** voient, les **alertes** Wazuh, puis le **verdict**. Le badge à côté de chaque titre rappelle le verdict. Les captures sont celles du dashboard et des terminaux pendant les tests.

Mise en place côté Windows

La VM Windows est sur le réseau isolé du lab : on n'y accède pas directement depuis le PC. On passe par un tunnel SSH via l'hôte Proxmox (`ssh -L 13389:10.10.10.20:3389 pve`), puis le client Bureau à distance se connecte à `localhost:13389`.

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

C:\Users\chef> ssh -L 13389:10.10.10.20:3389 pve
pve 7.0.2-6-pve #1 SMP PREEMPT_DYNAMIC PMX 7.0.2-6 (2026-05-20T08:55Z) x86_64

programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Sat Oct 3 19:30:48 2026 from 192.168.1.24
t@pve:~#
```

Figure 1 : Tunnel SSH vers la VM Windows

L'agent `win11` est actif et remonte ses logs au manager.

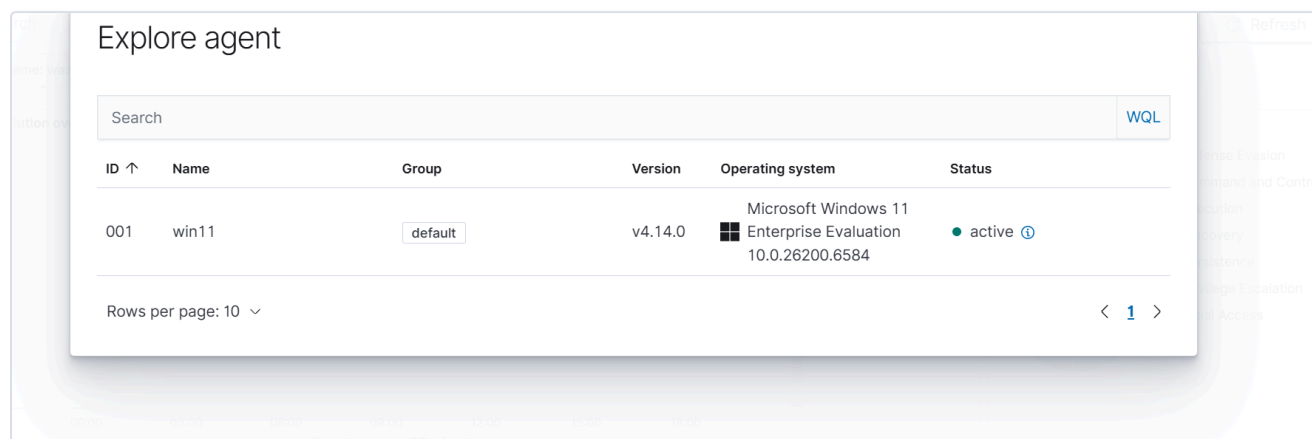


Figure 2 : Agent win11 actif

Sans aucune configuration, l'agent fait aussi un audit **SCA** contre le benchmark CIS Windows 11 (score de départ : 26 %) et de la **détection de vulnérabilités** sur les logiciels installés.

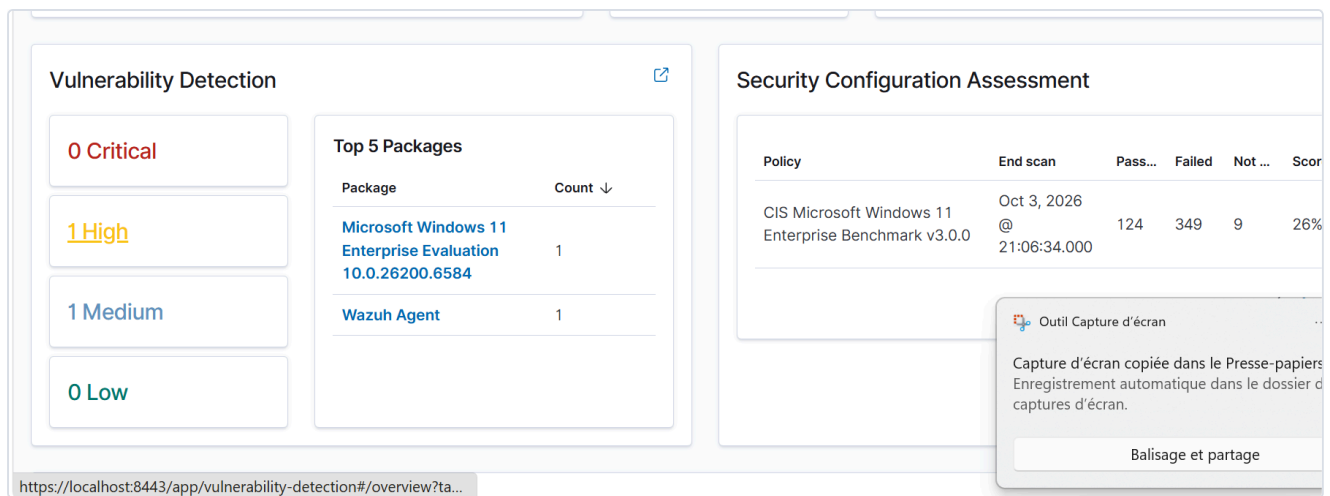


Figure 3 : SCA et vulnérabilités

Premier exemple de bruit : à chaque ouverture, PowerShell crée un fichier `__PSScriptPolicyTest_*` dans Temp (Sysmon event 11). C'est un comportement bénin, candidat à une exclusion.

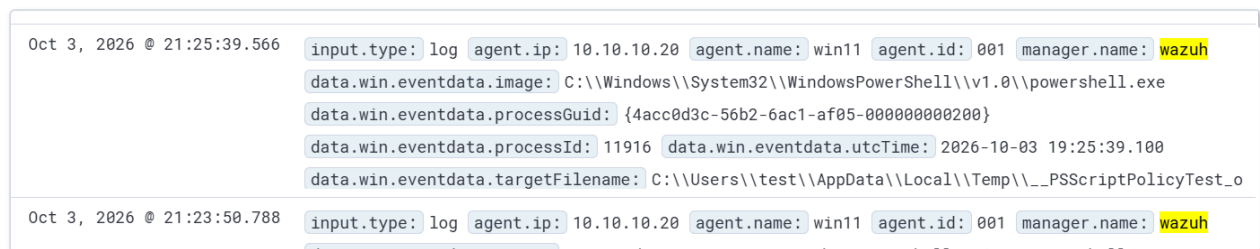


Figure 4 : Bruit Sysmon 11

Vue d'ensemble en fin de session : les alertes sont classées par sévérité selon le niveau de la règle (0-6 faible, 7-11 moyen, 12-14 haut, 15+ critique), et les tactiques MITRE vues sur `win11` apparaissent dans le panneau de l'agent.

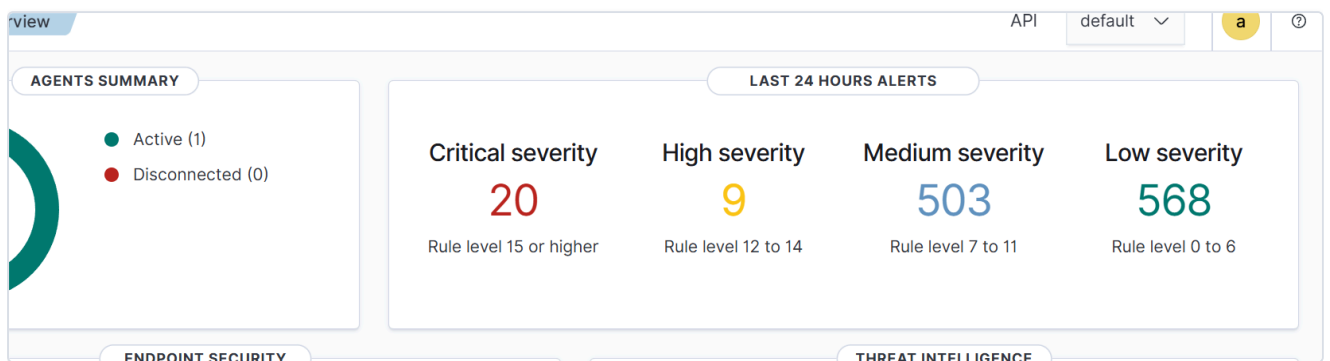


Figure 5 : Alertes par sévérité

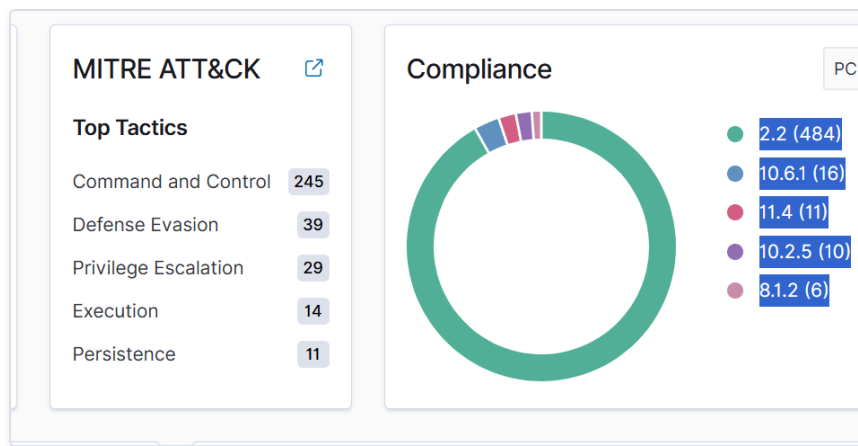


Figure 6 : Tactiques MITRE sur win11

1. T1136.001 : création d'un compte administrateur local Déecté

Test : Atomic `T1136.001-8`, qui lance `net user /add` puis `net localgroup administrators /add`. Côté attaquant, on liste d'abord les variantes disponibles (`-ShowDetailsBrief`) puis on lance la n°8 :

```
PS C:\WINDOWS\system32> Set-ExecutionPolicy Bypass -Scope Process -Force
PS C:\WINDOWS\system32> Import-Module C:\AtomicRedTeam\invoke-atomicredteam\Invoke-AtomicRedTeam.psd1 -Force
PS C:\WINDOWS\system32> Invoke-AtomicTest T1136.001 -ShowDetailsBrief
PathToAtomsFolder = C:\AtomicRedTeam\atoms

T1136.001-4 Create a new user in a command prompt
T1136.001-5 Create a new user in PowerShell
T1136.001-8 Create a new Windows admin user
T1136.001-9 Create a new Windows admin user via .NET
PS C:\WINDOWS\system32> Invoke-AtomicTest T1136.001 -TestNumbers 8
PathToAtomsFolder = C:\AtomicRedTeam\atoms

Executing test: T1136.001-8 Create a new Windows admin user
The command completed successfully.
The command completed successfully.
Exit code: 0
Done executing test: T1136.001-8 Create a new Windows admin user
PS C:\WINDOWS\system32>
```

Figure 7 : Lancement du test T1136.001-8

On vérifie sur le poste que le compte `T1136.001_Admin` existe bien (avant de le supprimer avec `-Cleanup`) :

```
Done executing test: T1136.001-8 Create a new Windows admin user
PS C:\WINDOWS\system32> net user

User accounts for \\DESKTOP-036DPFR

-----
Administrator          DefaultAccount          Guest
T1136.001_Admin         test                    WDAGUtilityAccount
The command completed successfully.
```

Figure 8 : Compte créé

Ce que les logs voient :

- journal Security : événements 4720 (compte créé) et 4732 (ajout au groupe Administrateurs) ;
- Sysmon : event 1, `net.exe` lancé par `cmd.exe`.

Alertes Wazuh : 3 alertes, dont une de niveau ≥ 12 (« Administrators Group Changed »). Elles sont rattachées à PCI DSS 10.2.5 et 8.1.2.

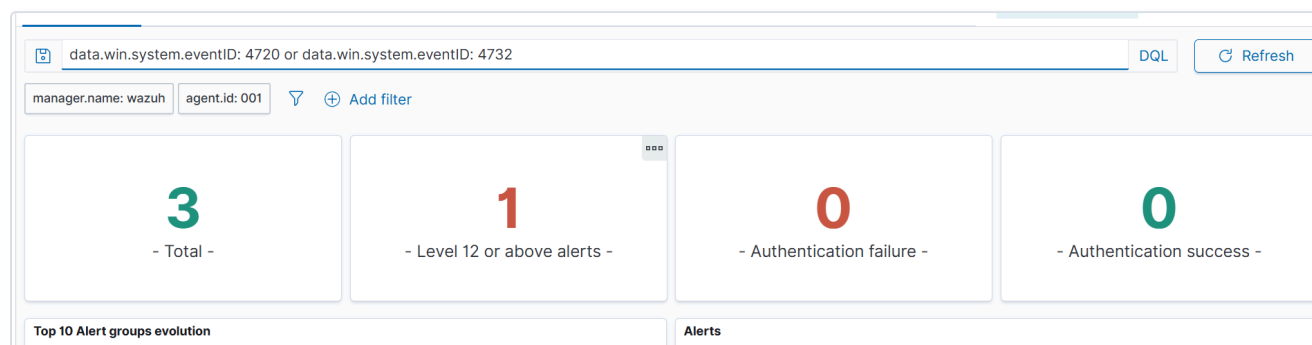


Figure 9 : Recherche 4720/4732

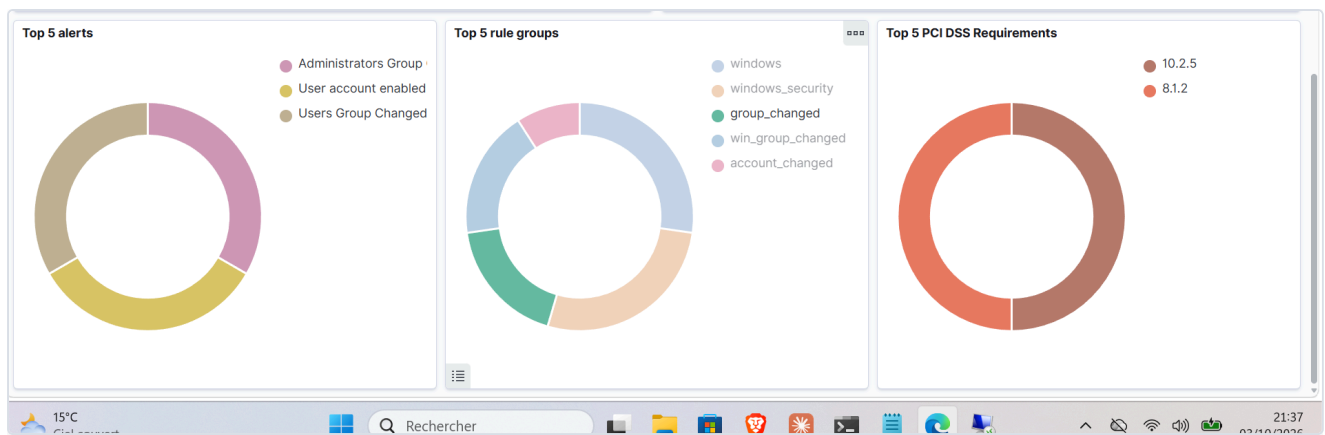


Figure 10 : Top alertes et PCI DSS

La commande elle-même est vue par Sysmon (« net.exe binary was started by a Windows cmd shell », groupe `sysmon_eid1_detections`) :

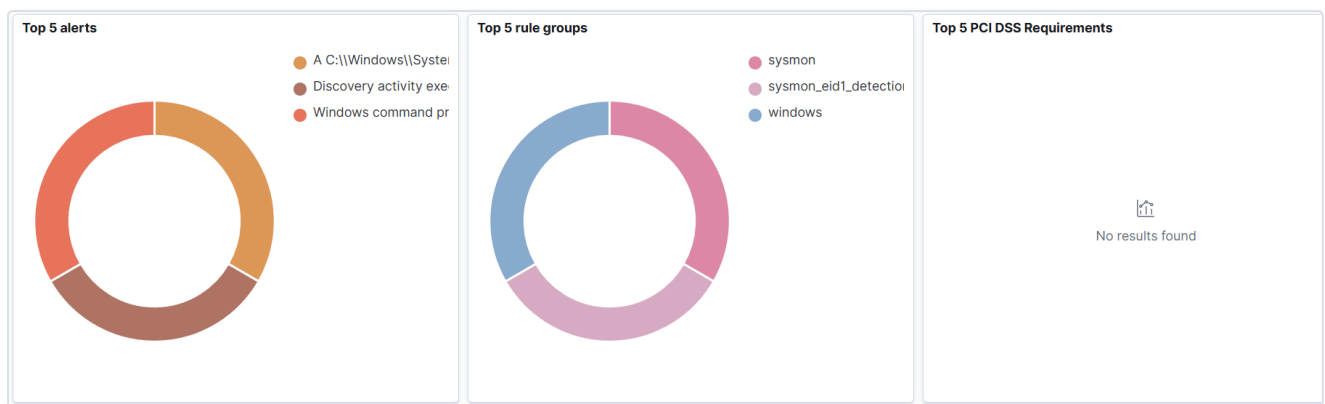


Figure 11 : net.exe vu par Sysmon

VERDICT

Détecté nativement, avec deux sources complémentaires. Le journal Security voit le **résultat** (compte, groupe), Sysmon voit la **commande**.

2. T1053.005 : tâche planifiée (règle personnalisée) Corrigé : règle 100100

Test : Atomic `T1053.005-2`, qui lance `SCHTASKS /Create /SC ONCE /TN spawn /TR cmd.exe`. La tâche « spawn » est bien créée :

```
PS C:\WINDOWS\system32>
PS C:\WINDOWS\system32> Invoke-AtomicTest T1053.005 -TestNumbers 2 -Cleanup
PathToAtomicsFolder = C:\AtomicRedTeam\atomics

Executing cleanup for test: T1053.005-2 Scheduled task Local
Done executing cleanup for test: T1053.005-2 Scheduled task Local
PS C:\WINDOWS\system32> Invoke-AtomicTest T1053.005 -TestNumbers 2
PathToAtomicsFolder = C:\AtomicRedTeam\atomics

Executing test: T1053.005-2 Scheduled task Local
SUCCESS: The scheduled task "spawn" has successfully been created.
WARNING: Task may not run because /ST is earlier than current time.
Exit code: 0
Done executing test: T1053.005-2 Scheduled task Local
PS C:\WINDOWS\system32>
```

Figure 12 : Lancement du test T1053.005-2

Ce que Sysmon voit (vérifié dans la VM avec `Get-WinEvent`) : event 1 `schtasks.exe`, avec la ligne de commande complète, le parent `cmd.exe`, le niveau d'intégrité High et les hashes.

PIÈGE RENCONTRÉ

La recherche `commandLine: *schtasks*` ne donnait rien, parce que la commande était en MAJUSCULES et que la recherche joker est sensible à la casse. Une recherche vide ne prouve pas l'absence d'un événement.

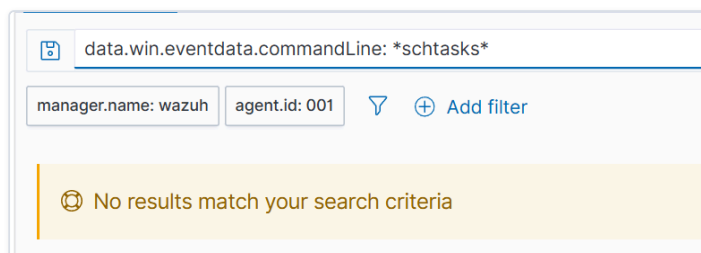


Figure 13 : Recherche vide à cause de la casse

Avant : en cherchant sur `image: *schtasks.exe`, on ne trouve qu'une alerte générique, la règle 92032 « Suspicious Windows cmd shell execution », de **niveau 3**. Elle est noyée dans le bruit, et rien n'y indique une persistance.

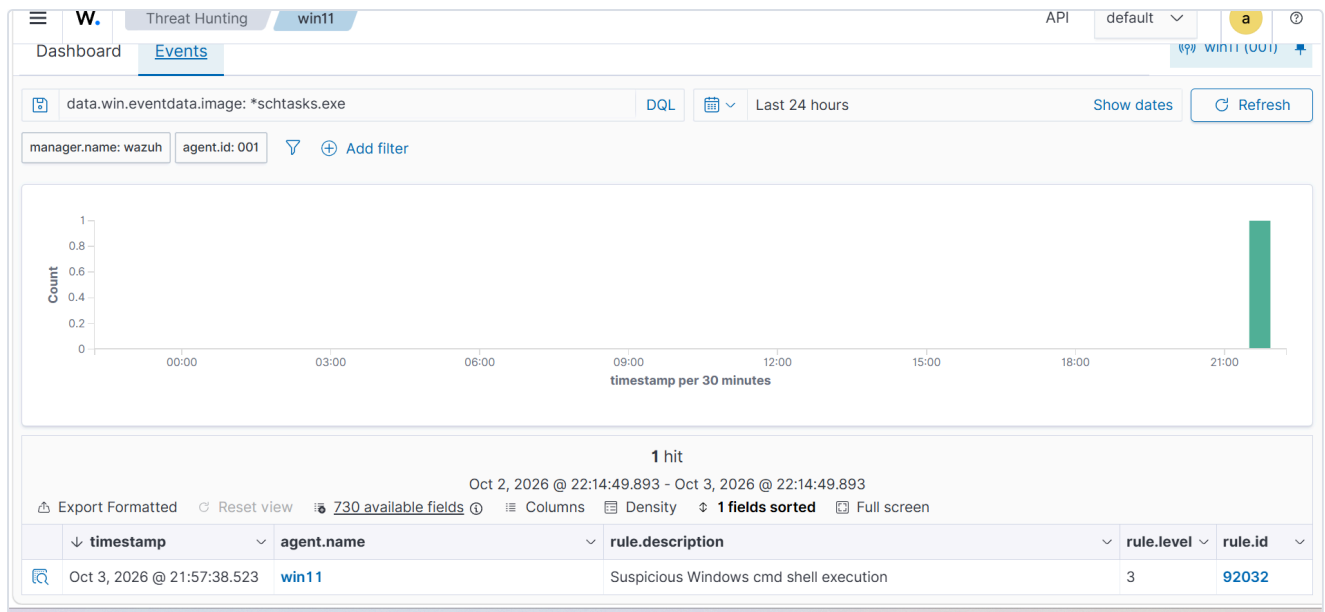


Figure 14 : Avant : 92032 niveau 3

Correction : règle `100100`, de niveau 10 avec le tag MITRE T1053.005. Elle se déclenche quand `schtasks.exe` est lancé avec `/create`, sans tenir compte de la casse (`(?i)`). Sa syntaxe a été validée avec `wazuh-analysisd -t` avant de redémarrer le manager.

Après : on rejoue le test. Dans le même tableau, on voit les deux exécutions : celle d'avant la règle (une seule ligne 92032 niveau 3) et celle d'après, où la règle 100100 niveau 10 apparaît **en plus** de la 92032, avec un titre explicite et le poste concerné. La nouvelle règle n'écrase rien, elle ajoute une alerte mieux qualifiée.

timestamp	agent.name	rule.description	rule.level	rule.id
Oct 3, 2026 @ 22:22:07.915	win11	Persistence : tache planifiee creee avec schtasks par DESKTOP-036DPF...	10	100100
Oct 3, 2026 @ 22:22:05.385	win11	Suspicious Windows cmd shell execution	3	92032
Oct 3, 2026 @ 21:57:38.523	win11	Suspicious Windows cmd shell execution	3	92032

Figure 15 : Après : règle 100100 niveau 10

POURQUOI NE PAS SIMPLEMENT MONTER LA 92032 ?

« cmd lance un programme » arrive des centaines de fois par jour : la monter créerait des faux positifs. On ajoute plutôt une règle **plus précise**, donc plus rare, qu'on peut mettre plus haut.

3. T1547.001 : clé de registre Run Détecté

Test : Atomic `T1547.001-1`, un `reg add` dans `...\CurrentVersion\Run`.

```
Executing test: T1547.001-1 Reg Key Run
The operation completed successfully.
Exit code: 0
Done executing test: T1547.001-1 Reg Key Run
PS C:\WINDOWS\system32>
```

Figure 16 : Lancement du test T1547.001-1

Alerte Wazuh : règle 92302, niveau 6, « Registry entry to be executed on next logon was modified using command line application reg.exe ». Elle porte le tag MITRE T1547.001 (Persistence, Privilege Escalation).

```
value
description: Registry entry to be 1
pending      executed on next (100%)
logon was modified
using command line
application reg.exe
```

Figure 17 : Alerte registre Run

timestamp	agent.name	rule.description	rule.level	rule.id
Oct 3, 2026 @ 22:24:37.928	win11	Registry entry to be executed on next logon was modified using comma...	6	92302

Figure 18 : Règle 92302 niveau 6

#	rule.level	6
	rule.mail	false
t	rule.mitre.id	T1547.001
t	rule.mitre.tactic	Persistence, Privilege Escalation
t	rule.mitre.technique	Registry Run Keys / Startup Folder

Figure 19 : Champs MITRE

En cliquant sur l'ID MITRE dans Wazuh, on ouvre la fiche de la technique : sa description, et ici les chemins de registre et dossiers de démarrage que les attaquants utilisent.

Details

ID	Name	Created Time	Modified Time	Version
T1547.001	Registry Run Keys / Startup Folder	Jan 23, 2020 @ 23:02:48.566	Mar 30, 2023 @ 23:01:52.183	1.2

Description

Adversaries may achieve persistence by adding a program to a startup folder or referencing it with a Registry run key. Adding an entry to the “run keys” in the Registry or startup folder will cause the program referenced to be executed when a user logs in.(Citation: Microsoft Run Key) These programs will be executed under the context of the user and will have the account’s associated permissions level.

Placing a program within a startup folder will also cause that program to execute when a user logs in. There is a startup folder location for individual user accounts as well as a system-wide startup folder that will be checked regardless of which user account logs in. The startup folder path for the current user is `C:\Users\[Username]\AppData\Roaming\Microsoft\Windows\Start Menu\Programs\Startup`. The startup folder path for all users is `C:\ProgramData\Microsoft\Windows\Start Menu\Programs\StartUp`.

The following run keys are created by default on Windows systems:

Figure 20 : Fiche MITRE T1547.001

VERDICT

Déecté et bien qualifié nativement. Pas besoin de règle perso : on ne complète que là où la détection native est insuffisante.

4. T1003.001 : dump de la mémoire de LSASS (trou de collecte) Corrigé : collecte Defender

Test : parmi les 14 variantes Atomic de T1003.001 (ProcDump, Mimikatz, NanoDump...), on a choisi la n°2, qui n'a besoin d'aucun outil externe.

```
PS C:\WINDOWS\system32> Invoke-AtomicTest T1003.001 -ShowDetailsBrief
PathToAtomicFolder = C:\AtomicRedTeam\atomics

T1003.001-1 Dump LSASS.exe Memory using ProcDump
T1003.001-2 Dump LSASS.exe Memory using comsvcs.dll
T1003.001-3 Dump LSASS.exe Memory using direct system calls and API unhooking
T1003.001-4 Dump LSASS.exe Memory using NanoDump
T1003.001-6 Offline Credential Theft With Mimikatz
T1003.001-7 LSASS read with pypykatz
T1003.001-8 Dump LSASS.exe Memory using Out-Minidump.ps1
T1003.001-9 Create Mini Dump of LSASS.exe using ProcDump
T1003.001-10 Powershell Mimikatz
T1003.001-11 Dump LSASS with createdump.exe from .Net v5
T1003.001-12 Dump LSASS.exe using imported Microsoft DLLs
T1003.001-13 Dump LSASS.exe using lolbin rdrleakdiag.exe
T1003.001-14 Dump LSASS.exe Memory through Silent Process Exit
PS C:\WINDOWS\system32>
```

Figure 21 : Variantes T1003.001

Atomic T1003.001-2, qui lance `rundll32.exe comsvcs.dll, MiniDump <PID lsass> lsass.dmp full`. Il s'agit d'un LOLBin : une DLL signée Microsoft détournée.

Ce qui s'est passé : « Access is denied ». Defender a bloqué la commande et l'a journalisée dans son propre canal : événement 1116 (Trojan:Win32/RundllLolBin.AF, Severe) puis 1117 (action Remove).

PROBLÈME

Rien dans Wazuh. L'agent ne collectait pas le canal `Microsoft-Windows-Windows Defender/Operational`.

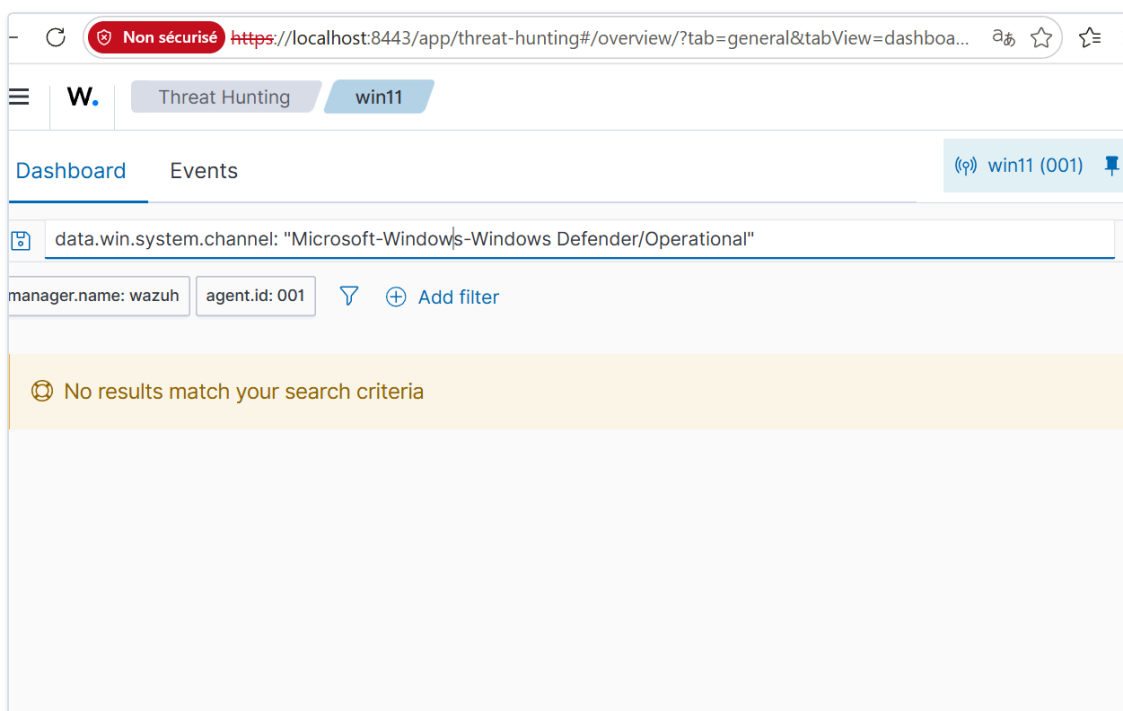


Figure 22 : Avant : canal Defender absent

Correction : ajout du canal dans `ossec.conf` de l'agent (`<localfile>` en `eventchannel`), puis on rejoue le test.

Après : 2 alertes, la détection et l'action, dont une de niveau ≥ 12 .

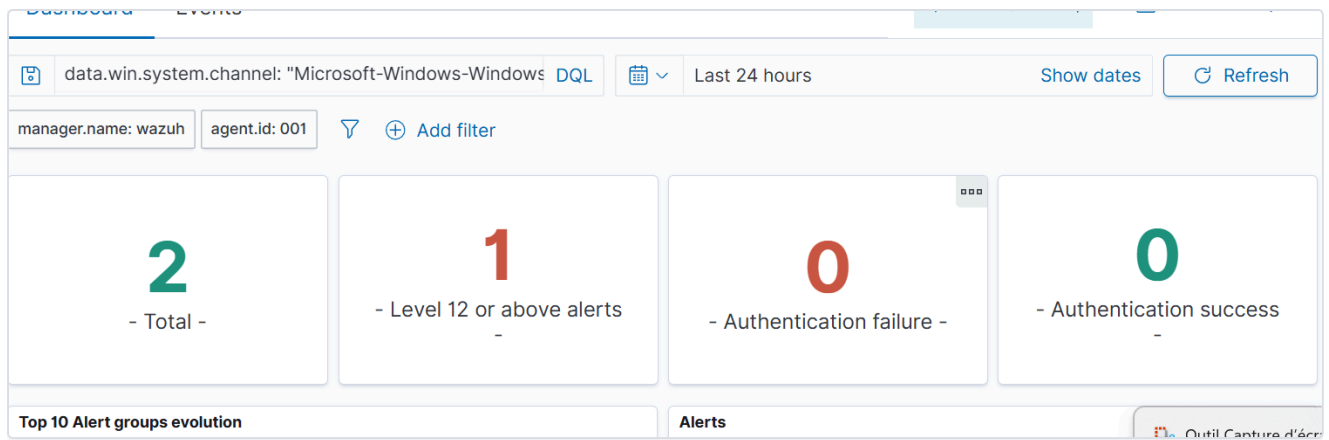


Figure 23 : Après : alertes Defender

rule.description: Descending	Count
Windows Defender: Antimalware platform detected potentially unwanted software ()	1
Windows Defender: Antimalware platform performed an action to protect you from potentially unwanted software ()	1

Figure 24 : Descriptions Defender

LEÇON

Une prévention doit aussi remonter au SOC. Une tentative bloquée veut dire qu'un attaquant est déjà sur le poste. **Limites restantes** : la description Wazuh n'affiche pas le nom de la menace (« () »), et la config SwiftOnSecurity ne journalise pas l'événement Sysmon 10 (accès à LSASS).

5. T1070.001 : effacement des journaux Détecté

Test : `wevtutil cl Security`, lancé à la main parce que le fichier YAML Atomic manquait dans le téléchargement.

Alerte Wazuh : « The audit log was cleared » (événement 1102).

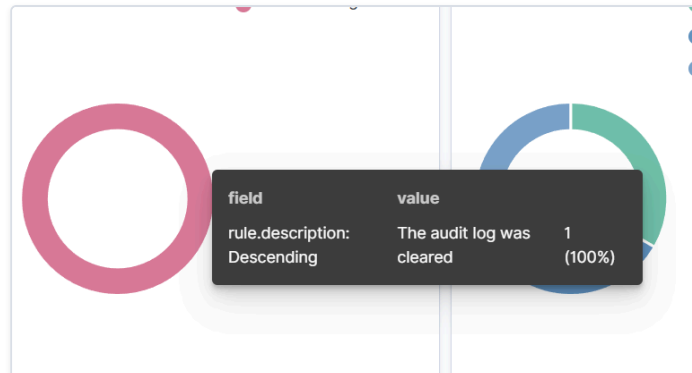


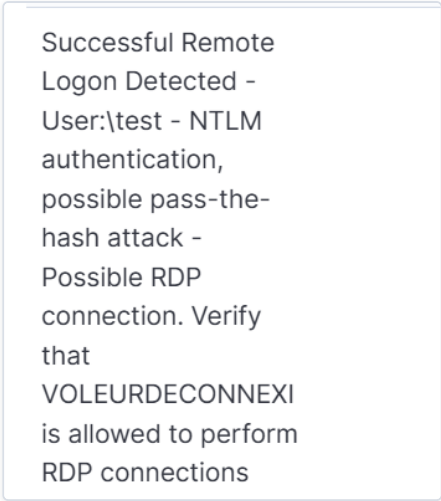
Figure 25 : Audit log cleared

LEÇON

Effacer les journaux sur le poste n'efface pas ce que le SIEM a déjà reçu. C'est une des raisons d'être d'un SIEM : centraliser les logs hors de portée de l'attaquant.

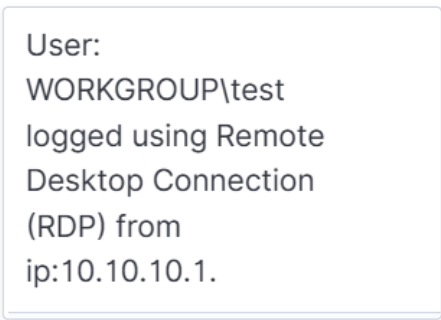
Bonus : qualification d'un faux positif Faux positif qualifié

Wazuh a signalé « Successful Remote Logon... NTLM authentication, possible pass-the-hash attack ». L'enquête montre que l'IP source est 10.10.10.1 (le bastion Proxmox, via le tunnel SSH), que le poste source est le mien, et que l'heure correspond à ma connexion RDP. C'est donc une activité d'administration légitime. En production, on documenterait l'exception ou on ajusterait la règle pour les IP d'administration connues.



Successful Remote Logon Detected - User:\test - NTLM authentication, possible pass-the-hash attack - Possible RDP connection. Verify that VOLEURDECONNEXI is allowed to perform RDP connections

Figure 26 : Faux positif RDP NTLM



User: WORKGROUP\test logged using Remote Desktop Connection (RDP) from ip:10.10.10.1.

Figure 27 : IP source 10.10.10.1

Linux : ubuntu-cible

L'agent 002 est actif. Il lance tout seul l'audit SCA CIS Ubuntu 24.04 et lit journald (sshd, sudo).

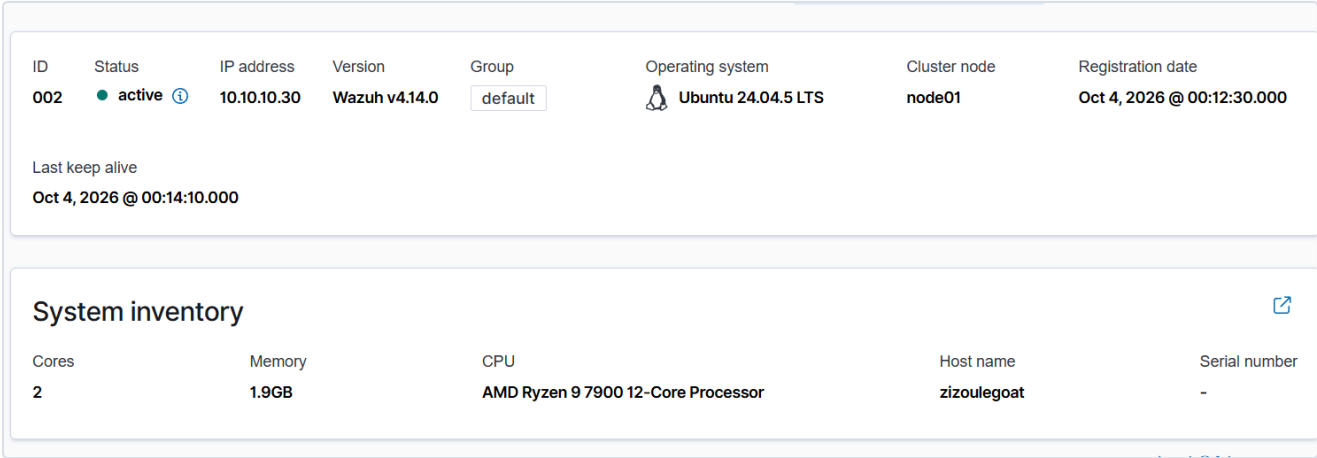


Figure 28 : Agent Ubuntu actif

L1. T1110 : brute force SSH Détecté

Test : 10 connexions SSH avec des utilisateurs inexistants (pirate1 à pirate10).

Résultat : une alerte sshd par tentative. Le panneau MITRE de l'agent affiche Credential Access : 10.

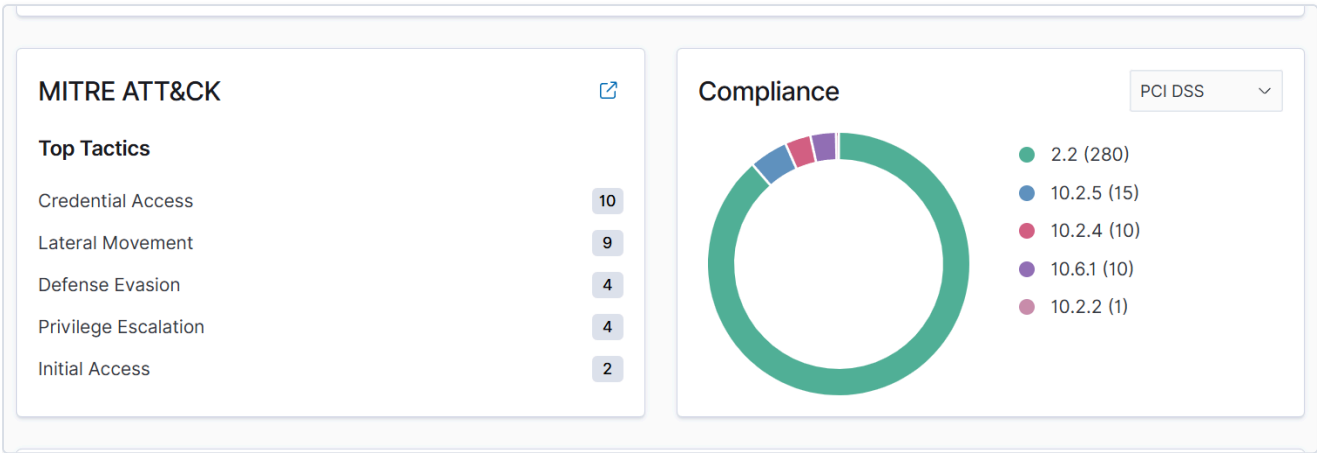


Figure 29 : MITRE Ubuntu

L2. FIM : création d'un compte vue par le contrôle d'intégrité Détecté

Configuration : par défaut, l'agent scanne /etc toutes les 12 heures. On le passe en realtime="yes" avec report_changes="yes".

Test : useradd -m backdoor, puis userdel -r backdoor pour nettoyer.

Résultat : règle 550 « Integrity checksum changed », niveau 7, 12 fois (passwd, shadow, group, gshadow, subuid, subgid et leurs sauvegardes -). S'y ajoutent « File added / deleted » pour les fichiers temporaires et les verrous. Chaque alerte donne le fichier, la taille, la date de modification et les hashes avant et après.

agent.id	002
agent.ip	10.10.10.30
agent.name	ubuntu-cible
decoder.name	syscheck_integrity_changed
full_log	File '/etc/subuid-' modified Mode: realtime Changed attributes: size,mtime,md5,sha1,sha256 Size changed from '0' to '29' Old modification time was: '1788935406', now it is '179104913' Old md5sum was: 'dA1d8cd98f00h20Ae9800998eef8A127e'
id	1791066525.6188870
input.type	log

Figure 30 : Détail FIM /etc/subuid-

	timestamp	agent.name	rule.description	rule.level	rule.id
	Oct 4, 2026 @ 00:28:45.525	ubuntu-cible	Integrity checksum changed.	7	550
	Oct 4, 2026 @ 00:28:45.525	ubuntu-cible	Integrity checksum changed.	7	550
	Oct 4, 2026 @ 00:28:45.525	ubuntu-cible	Integrity checksum changed.	7	550
	Oct 4, 2026 @ 00:28:45.505	ubuntu-cible	Integrity checksum changed.	7	550
	Oct 4, 2026 @ 00:28:45.505	ubuntu-cible	Integrity checksum changed.	7	550

Figure 31 : Règle 550

rule.description: Descending		Count
Integrity checksum changed.		12
File added to the system.		3
File deleted.		3
Rows per page: 20		<

Figure 32 : Compteurs FIM

CHOIX ASSUMÉ

Le temps réel est plus réactif, mais il coûte plus de ressources et génère plus de bruit qu'un scan périodique.

6 Leçons et recommandations

6.1 Leçons techniques

Constat	Leçon
5 tests sur 7 détectés sans rien modifier	Mesurer la détection native avant d'écrire des règles. On ne complète que là où il y a un trou, sinon on multiplie les alertes en double.
Tâche planifiée vue en niveau 3 seulement	Une détection peut exister et être inutile en pratique si son niveau la noie dans le bruit. La bonne correction est une règle plus précise , pas une règle générique remontée.
Blocage par Defender invisible pour le SIEM	Toute source de sécurité doit remonter au SOC, y compris les préventions réussies : une tentative bloquée signifie que quelqu'un est déjà sur le poste.
Recherches vides alors que l'événement existait	Avant de conclure « non détecté », vérifier la casse, les filtres restés actifs, la période, et se rappeler que seuls les événements qui déclenchent une règle sont indexés.
Journaux effacés sur le poste mais conservés dans le SIEM	C'est la raison d'être d'un SIEM : garder une copie des logs hors de portée de l'attaquant.
Alerte « pass-the-hash » sur ma propre connexion RDP	Un analyste qualifié avant d'escalader : IP source, poste, heure, contexte. Un faux positif connu se documente ou s'exclut précisément.

6.2 Recommandations pour un environnement réel

1. **Collecter systématiquement** les canaux de sécurité Windows au-delà de Security et Sysmon (Defender, PowerShell, tâches planifiées), et le vérifier par un test, pas seulement par la configuration.
2. **Tester chaque règle** par une émulation avant et après sa mise en production, comme ici pour la règle 100100.
3. **Documenter les exclusions** (administrateurs connus, bruit PowerShell) au lieu de baisser les niveaux globalement.
4. **Rejouer les tests régulièrement** : une mise à jour de Wazuh, de Sysmon ou de sa configuration peut faire disparaître une détection sans prévenir.

7 Limites et prochaines étapes

7.1 Limites de cet exercice

- **Peu de tests par technique** : une seule variante Atomic par technique. Un attaquant réel peut obtenir le même résultat par d'autres moyens qui ne déclencheraient pas les mêmes règles.
- **Accès à LSASS non journalisé** : la configuration Sysmon utilisée n'enregistre pas l'événement 10 (accès à un processus). Sans Defender, la tentative du test 4 aurait été invisible.
- **Nom de la menace absent** dans la description des alertes Defender (« () ») : il faudrait adapter le décodeur ou la règle.
- **Pas de visibilité réseau** : toutes les détections viennent des machines elles-mêmes. Un scan ou une connexion suspecte entre machines ne serait pas vu.
- **Lab simplifié** : un seul réseau à plat, pas de domaine Active Directory, très peu de trafic normal, donc peu de bruit par rapport à une vraie entreprise.

7.2 Prochaines étapes prévues

Étape	Ce qu'elle apporte
Suricata sur un Raspberry Pi 5	Détection réseau (IDS) : scans, connexions vers des destinations suspectes. Alertes envoyées à Wazuh.
Pare-feu OPNsense et VLAN	Segmenter le lab (administration, serveurs, cibles) et envoyer les logs du pare-feu au SIEM.
Sysmon : activer l'événement 10 sur LSASS	Voir les tentatives d'accès à la mémoire des identifiants même sans antivirus.
Automatisation (Ansible, Terraform)	Reconstruire le lab et redéployer les règles en une commande, et rejouer les tests automatiquement.

A Annexe A : glossaire

Terme	Définition
Agent	Programme installé sur une machine pour envoyer ses journaux au SIEM.
Alerte	Événement qui a déclenché une règle du SIEM ; elle porte un niveau, une description et souvent un tag MITRE.
Atomic Red Team	Bibliothèque open source de tests qui reproduisent chacun une technique ATT&CK.
CVE	Identifiant public d'une vulnérabilité logicielle précise.
Décodeur	Partie du SIEM qui découpe un log brut en champs exploitables.
Faux positif	Alerte levée sur une activité légitime.
FIM	File Integrity Monitoring : surveillance des modifications de fichiers sensibles par comparaison d'empreintes (hash).
Hash	Empreinte numérique d'un fichier ; si un octet change, l'empreinte change.
LOLBin	« Living off the land binary » : programme légitime du système détourné par un attaquant pour ne pas apporter ses propres outils.
LSASS	Processus Windows qui gère l'authentification ; sa mémoire contient des secrets d'identification, d'où l'intérêt des attaquants.
MITRE ATT&CK	Base de connaissances des tactiques et techniques d'attaquants observées dans la réalité.
Niveau (Wazuh)	Gravité d'une règle, de 0 à 15.
PCI DSS	Norme de sécurité des données de cartes bancaires ; Wazuh rattache certaines alertes à ses exigences.
Purple team	Exercice où l'on joue à la fois l'attaque (red) et la défense (blue) pour améliorer la détection.
SCA	Security Configuration Assessment : audit de configuration d'une machine par rapport à un référentiel (ici CIS).
SIEM	Security Information and Event Management : outil qui centralise les logs et lève des alertes.
Sysmon	Outil Microsoft qui ajoute des journaux détaillés à Windows (processus, fichiers, registre, réseau).
Tactique / technique	Dans ATT&CK, la tactique est le but de l'attaquant (persister, voler des identifiants) ; la technique est le moyen.

B Annexe B : la règle 100100 expliquée ligne par ligne

Fichier `rules/local_rules.xml` sur le manager Wazuh. Les règles personnalisées utilisent les identifiants à partir de 100000 pour ne jamais entrer en conflit avec celles livrées par Wazuh.

```
<group name="local,sysmon,persistence,">
  <rule id="100100" level="10">
    <if_sid>61603, 92032</if_sid>
    <field name="win.eventdata.image" type="pcre2">(?!i)schtasks\.exe$</field>
    <field name="win.eventdata.commandLine" type="pcre2">(?!i)/create</field>
    <description>Persistance : tâche planifiée créée avec schtasks par $(win.eventdata.user)
  </description>
  <mitre><id>T1053.005</id></mitre>
</rule>
</group>
```

Élément	Rôle
<code>group name</code>	Étiquettes qui permettent de filtrer ces alertes dans le dashboard.
<code>id="100100"</code> <code>level="10"</code>	Identifiant unique de la règle, et niveau 10 : sévérité moyenne haute, au-dessus du bruit.
<code>if_sid</code>	La règle n'est évaluée que si l'événement a déjà déclenché la règle 61603 (création de processus Sysmon) ou 92032 (programme lancé par cmd). C'est un raffinement de règles existantes.
<code>field ... image</code>	Le programme lancé doit être <code>schtasks.exe</code> . <code>(?!i)</code> rend la comparaison insensible à la casse, <code>\$</code> impose la fin du chemin.
<code>field ... commandLine</code>	La commande doit contenir <code>/create</code> : on ne veut que les créations de tâche, pas les simples consultations.
<code>description</code>	Titre lisible de l'alerte ; <code>\$(win.eventdata.user)</code> y insère l'utilisateur concerné.
<code>mitre</code>	Rattache l'alerte à la technique T1053.005 pour les tableaux de bord MITRE.

BONNE PRATIQUE APPLIQUÉE

Avant de redémarrer le manager, la syntaxe a été vérifiée avec l'outil de test de Wazuh (`wazuh-analysisd -t`). Une erreur dans ce fichier empêche le manager de démarrer, et donc coupe toute la détection.

C Annexe C : réutiliser ce rapport comme modèle

Ce document suit la structure classique d'un rapport technique d'exercice. Pour un nouvel exercice (TP, audit, autre lab), garder le même squelette :

Partie	Ce qu'on y met	Lecteur visé
Page de garde	Titre, auteur, date, version, périmètre, niveau de diffusion	Tous
Résumé	La question posée, ce qui a été fait, les chiffres clés, le tableau des résultats, 3 messages à retenir. Une page maximum.	Manager, recruteur
Contexte et notions	Le vocabulaire nécessaire pour comprendre la suite	Débutant
Périmètre et architecture	Les machines, le réseau, un schéma, ce qui est hors périmètre	Technique
Méthodologie	Les étapes suivies, la grille de verdict, les règles du jeu (autorisations, nettoyage)	Technique, auditeur
Résultats détaillés	Une fiche par test, toujours dans le même ordre : action, observation, preuve (capture), verdict	Technique
Leçons et recommandations	Ce qu'on a appris, ce qu'on conseille de faire	Tous
Limites et suite	Ce que le test ne prouve pas, et ce qui vient après. Être honnête ici rend le reste crédible.	Tous
Annexes	Glossaire, code, configurations, détails trop longs pour le corps	Technique

RÈGLES DE FORME À GARDER

- Chaque affirmation a une preuve : une capture numérotée, une sortie de commande, un identifiant de règle.
- Un verdict par test, avec une grille définie à l'avance.
- Les captures ont une légende qui dit ce qu'il faut y regarder.
- Jamais de mot de passe, de clé ou d'adresse publique dans un document diffusé.
- Le résumé s'écrit en dernier, mais se place en premier.

Les sources de ce document (HTML, CSS, script de génération) sont dans le dossier `rapport/` du dépôt : modifier `detections/README.md` puis relancer `python3 build.py` régénère le PDF.